

High-Dimensional Indexing for Efficient Multimedia Retrieval using R-Tree

Team Name : MaiSQL

Members:

Shaik Asim Shareef – 23CS10065

Pardha Saradhi – 23CS10016

Vinay Koduru – 23CS10079

M Lekhya Reddy – 23CS10039

Y. Lavanya Vani – 23CS10082

1 Objective

The primary objective of this project is to design and implement a custom spatial database engine capable of performing K-Nearest Neighbor (KNN) query execution on multimedia data (audio and images).

Traditional indexing structures in relational databases, such as B+ Trees, rely on a 1-dimensional total ordering of keys and are entirely inadequate for indexing multi-dimensional spatial data. To bridge this gap, this project aims to:

1. Implement custom **KD-Tree** and **R-Tree** spatial access methods from database first principles, managing how high-dimensional vectors are clustered and partitioned.
2. Mitigate the “Curse of Dimensionality” using Principal Component Analysis (PCA) as a data compression technique, reducing vector payload sizes to increase the index branching factor (fanout).
3. Rigorously compare the query execution cost (evaluating both CPU latency and simulated Disk I/O node fetches) of the implemented Index Scans against a baseline Sequential Scan (Full Table Scan) across varying database sizes (N), dimensions (D), and query radii (K).

2 Architecture and Implementation

2.1 Data Ingestion and Vector Schema

In a traditional DBMS, data is stored as scalar types. To index multimedia, files must be transformed into continuous mathematical vectors (BLOB metadata) before insertion into the database.

- **Image Processing:** Images are passed through a Convolutional Neural Network (CNN) to extract deep semantic embeddings, resulting in dense floating-point vector attributes.
- **Audio Processing:** Audio waveforms are processed to extract Mel-Frequency Cepstral Coefficients (MFCCs), forming a mathematical signature of the acoustic data.

2.2 Index Optimization via Dimensionality Reduction

Spatial access methods suffer from severe performance degradation in hyper-dimensional spaces ($D > 20$). To optimize the index, the ingested feature vectors are compressed using Principal Component Analysis (PCA). By projecting the vectors onto orthogonal axes of maximum variance, we reduce the schema to an optimal D -dimensions.

In a DBMS context, this acts as a critical payload compression step, allowing more records to fit inside a single disk page or node, thereby reducing tree height and I/O costs.

2.3 KD-Tree Access Method

The KD-Tree is implemented as an in-memory binary space-partitioning index structure.

- **Recursive Splitting:** At each level of the index, the query optimizer selects a specific attribute (axis) to partition the data:

$$\text{axis} = \text{depth} \% D$$

- **Median Pivot:** To guarantee a perfectly balanced index and predictable query times, the dataset is sorted along the chosen axis, and the median record is selected as the partition boundary.
- **Partitioning:** Records strictly less than the median are routed to the left subtree, while the rest are routed right, effectively halving the search space per level.

2.4 R-Tree Access Method (Disk-Oriented Approach)

Unlike the KD-Tree, the R-Tree is a dynamic, balanced, multi-way tree designed similarly to a B-Tree, heavily optimized to minimize disk page fetches by grouping data into overlapping Minimum Bounding Rectangles (MBRs).

- **Node Serialization Strategy:** Leaf nodes contain the actual vector records (or pointers to the disk block). Internal nodes contain MBR coordinates ($X_{\min}, X_{\max}$) and pointers to child nodes. The maximum node capacity (M) is traditionally tuned to match the operating system's disk block size (e.g., 4KB).
- **Insertion (*ChooseLeaf*):** During an INSERT operation, the engine traverses from the root, calculating the area enlargement required for each child's MBR. The record is routed to the child requiring minimum spatial expansion.

- **Page Splitting (Quadratic Split):** When a node exceeds capacity M , a page split occurs. Since finding the optimal split is $\mathcal{O}(2^M)$, we implemented a Quadratic Split heuristic ($\mathcal{O}(M^2)$):
 1. **PickSeeds:** Evaluates every pair of records to find the two that would waste the most area if kept in the same page.
 2. **PickNext:** Iteratively assigns remaining records to the page that requires the least MBR enlargement.

2.5 Query Execution Engine (KNN Search)

To execute similarity queries, we implemented an optimized Branch-and-Bound traversal mimicking a database Index Scan, utilizing a Min-Priority Queue.

- **MinDist Metric:** Calculates the shortest possible distance between the query vector and an MBR boundary.
- **Traversal Logic:** Nodes with the smallest MinDist are prioritized.
- **Index Pruning:** Nodes with MinDist greater than the K -th closest result are pruned.

2.6 Performance Evaluation Metrics

- **Sequential Scan Baseline:** Linear search with $\mathcal{O}(N)$ complexity.
- **Cost Metrics:** Wall-clock time and total nodes visited.
- **Stress Testing:** Evaluated across varying N , D , and K values.

3 Results and System Administration Interface

3.1 Main Dashboard



Figure 1: The administration dashboard of the Spatial Database Engine.

3.2 Multimedia Query Interface

← Back to DBMS Dashboard

DBMS Multimedia KNN Search

1. Database Index

Images

Audio

2. Target Neighbors (K)

5

3. Upload Query Signature

Choose File

cat-with-white-and-golden-color-on-blurred-brown-background-photo.webp

⚡ EXECUTE TRAVERSAL

🗺 View Map

Retrieved Nearest Neighbors:



Rank #1
57.jpg
Dist: 3.9226



Rank #2
1051.jpg
Dist: 4.0203



Rank #3
13.jpg
Dist: 4.0407



Rank #4
486.jpg
Dist: 4.2134



Rank #5
371.jpg
Dist: 4.5850

Figure 2: Image-based KNN search results.

[← Back to DBMS Dashboard](#)

DBMS Multimedia KNN Search

1. Database Index

 Images

 Audio

2. Target Neighbors (K)
5

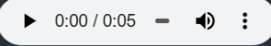
3. Upload Query Signature

Choose File

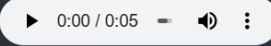
1-137-A-32.wav

⚡ EXECUTE TRAVERSAL

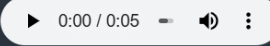
 View Map



Rank #1
1-137-A-32.wav
Dist: 0.0000



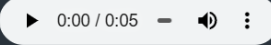
Rank #2
3-94355-A-33.wav
Dist: 0.0544



Rank #3
5-235593-A-23.wav
Dist: 0.0579



Rank #4
1-17092-A-27.wav
Dist: 0.0586



Rank #5
2-95258-B-1.wav
Dist: 0.0599

Figure 3: Audio-based KNN search results.

3.3 Interactive Index Visualizer

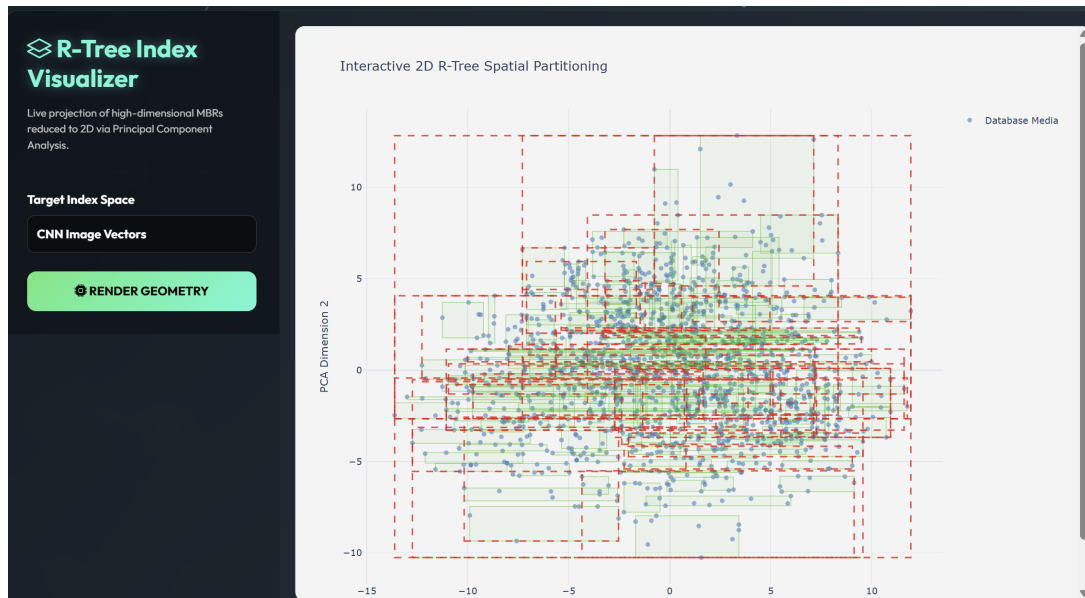


Figure 4: 2D projection of R-Tree spatial partitions.

3.4 Query Performance Analytics



Figure 5: Performance comparison across varying dimensions.

4 References

1. GeeksforGeeks. *K-Dimensional Tree (kd tree) - Data Structure and Algorithm Tutorials*. <https://www.geeksforgeeks.org/k-dimensional-tree/>
2. GeeksforGeeks. *Introduction to R-Tree*. <https://www.geeksforgeeks.org/introduction-to-r-t>
3. Stack Overflow. *Algorithm for finding K-Nearest Neighbors efficiently in high-dimensional space*. <https://stackoverflow.com/questions/1402014/algorithm-for-finding-k-nearest>
4. Towards Data Science. *The Curse of Dimensionality: A Gentle Introduction*. <https://towardsdatascience.com/the-curse-of-dimensionality-50ba6e9a6ea6>
5. Scikit-learn Developers. *Principal Component Analysis (PCA)*. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
6. Librosa Development Team. *MFCC Feature Extraction*. <https://librosa.org/doc/latest/generated/librosa.feature.mfcc.html>
7. Kaggle Dataset. *Microsoft Cats vs Dogs Dataset*. <https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset>
8. UrbanSound8K Dataset. <https://www.kaggle.com/datasets/chrisfilo/urbansound8k>